MLOps Lab Report

Student: Dhanya R ID: 24AI021
Course: SEL Lab
Generated: 06 Nov 2026 22:40
GitHub: https://github.com/DhanyaR-07

1. Objectives

Refactor the house-price ML project into modules (data, train, evaluate, predict), serve the saved scikit-learn Pipeline through FastAPI (GET /health, POST /predict) and package it with Docker/Compose so another developer can reproduce it.

2. Architecture

Data -> Cleaning -> Feature Engineering (imputer + scaler) -> Training (RandomForest) -> Evaluation -> Saved Pipeline (models/model.joblib) -> FastAPI POST /predict -> Docker container.

3. Pipeline stages (input / output / purpose / failure)

Loading: file path -> DataFrame; fails if file/columns missing.
Cleaning: raw DataFrame -> clean DataFrame; fails if valid rows are dropped wrongly.
Feature engineering: columns -> model-ready features; fails on train/inference mismatch.
Training: X_train,y_train -> fitted Pipeline; fails by overfitting or data leakage.
Evaluation: held-out data -> metrics; fails if test data leaks into training.
Prediction: new records -> predictions; fails on wrong feature names/order.

4. Online vs batch inference

Online: one/few records, immediate response, latency matters (POST /predict). Batch: many records or a CSV, scheduled or background job, throughput matters. For a CSV I would add an upload endpoint that queues a background job and writes a results file, or run a separate scheduled batch script using the same pipeline.

5. Docker concepts

Works on one laptop but not another because of different OS, Python and library versions. Containers ship the same runtime and dependencies. They do not guarantee identical hardware, performance, security, or that data/models are present (host paths are not visible inside a container unless mounted).

6. Implementation status and evidence

[PASS] Data download

```
$ python -m src.data
Raw dataset saved to data/raw/housing.csv (20640 rows)
```

[PASS] Training

```
$ python -m src.train
Model saved to models/model.joblib
Test RMSE=0.5443  R2=0.7739
MLflow not installed; skipping tracking.
```

[PASS] Evaluation

```
$ python -m src.evaluate
{'rmse': 0.5443055334680222, 'mae': 0.36632235428263393, 'r2': 0.7739114133958234, 'n_test_rows': 4128}
```

[PASS] Prediction (separate process)

```
$ python -m src.predict
[4.348756928267501]
```

[PASS] Tests (pytest)

```
$ python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\dhanya.r\OneDrive\Downloads\24AI021_ML_Lab-
cachedir: .pytest_cache
rootdir: C:\Users\dhanya.r\OneDrive\Downloads\24AI021_ML_Lab-6,7
plugins: anyio-4.16.0
collecting ... collected 5 items
```

```
tests/test_api.py::test_health_and_valid_prediction PASSED               [ 20%]
tests/test_api.py::test_invalid_requests PASSED                          [ 40%]
tests/test_pipeline.py::test_preprocess_removes_invalid_and_duplicates PASSED [ 60%]
tests/test_pipeline.py::test_predict_output_shape PASSED                 [ 80%]
tests/test_pipeline.py::test_predict_missing_feature_raises PASSED       [100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\dhanya.r\OneDrive\Downloads\24AI021_ML_Lab-6,7\.venv\Lib\site-packages\fastapi\testclient.py:1: Starlett
    from starlette.testclient import TestClient as TestClient  # noqa
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 5 passed, 1 warning in 2.40s =========================
```

[FAIL / NOT COMPLETED] Docker images

```
$ docker image ls
failed to connect to the docker API at npipe:////./pipe/docker_engine; check if the path is correct and if the daem
```

[FAIL / NOT COMPLETED] Docker containers

```
$ docker ps -a
failed to connect to the docker API at npipe:////./pipe/docker_engine; check if the path is correct and if the daem
```

[FAIL / NOT COMPLETED] Docker logs

```
$ docker logs ml-api
failed to connect to the docker API at npipe:////./pipe/docker_engine; check if the path is correct and if the daem
```

7. API validation (FastAPI TestClient)

```
Health check            200     {"status":"ok","model_loaded":true}
Valid input             200     {"prediction":[4.348756928267502],"unit":"100k USD"}
Missing field           422     {"detail":[{"type":"missing","loc":["body","MedInc"],"msg":"Field requ
Wrong type              422     {"detail":[{"type":"float_parsing","loc":["body","HouseAge"],"msg":"In
Invalid business value  422     {"detail":[{"type":"greater_than","loc":["body","MedInc"],"msg":"Input
```

8. Results


```
{
  "rmse": 0.5443055334680222,
  "mae": 0.36632235428263393,
  "r2": 0.7739114133958234,
  "n_test_rows": 4128
}
```


9. Screenshots

0 screenshot(s) attached on the following pages.

10. Challenges and resolutions

- Problem: Docker Desktop could not run: 'The paging file is too small for this operation to complete' (Windows virtual memory exhausted), so docker build/run was not executed.
  Resolution: Not resolved in the time available. Dockerfile, .dockerignore and docker-compose.yml are written and included as untested work; the fix would be to increase the Windows paging file / free RAM and rerun.
- Problem: OpenBLAS memory allocation error while running numpy/scikit-learn.
  Resolution: Set OPENBLAS_NUM_THREADS=1 and ran uvicorn without --reload.
- Problem: Docker Desktop not found at the default path, and PowerShell vs cmd syntax differences.
  Resolution: Located the install under AppData\Local\Programs\DockerDesktop and used PowerShell syntax.

11. Reflection

I learned how to separate ML code into modules, save one Pipeline for both training and inference, validate API inputs with Pydantic, and package the service with Docker. Items marked FAIL above were not completed in the available time and are listed honestly.